

第 5 章 模块和第三方库

前面写的所有代码都是自己从头敲的。实际做事不必这样，很多常见需求别人早就写好了，拿来用就行。这一章统计一段文字里的高频词，全程借用现成的工具，顺便搞清楚这些工具从哪来、怎么装。

5.1 先统计一段文字的高频词

本节任务：给一段英文，找出出现次数最多的几个词，并统计一共有多少个不重复的词。

```
1 import re
2 from collections import Counter
3
4 text = """Python is easy to read and easy to write.
5 Python is widely used in data analysis and machine learning.
6 Read the code, write the code, and read it again."""
7
8 words = re.findall(r"[a-z]+", text.lower()) # 全转小写，只取字母组成的词
9 counter = Counter(words) # 一行完成计数
10
11 print("总词数: ", len(words))
12 print("不重复的词: ", len(counter))
13 print("出现最多的五个: ")
14 for word, n in counter.most_common(5):
15     print(f" {word} {n} 次")
```

运行结果：

```
总词数:  29
不重复的词:  18
出现最多的五个:
  read 3 次
  and 3 次
python 2 次
is 2 次
easy 2 次
```

注意这段代码里真正属于我们的逻辑只有两三行。切词交给了 `re`，计数交给了 ★核心 `Counter`，找前五名用的是 `Counter` 自带的 `most_common`。如果全都自己写，光计数就

得建字典、循环、判断键在不在，排序还得再写一段。**善用现成的工具**是写代码效率的分水岭。

下面讲清楚这些工具是怎么拿到的。

5.2 import 的几种写法

本节任务：会把别人写好的代码引进自己的程序。

一个 py 文件就是一个**模块**。用 **import** 可以把别的模块引进来用，有三种常见写法：

```
1 import math # 引入整个模块，用的时候带模块名
2 print(math.sqrt(16)) # 4.0
3
4 from math import sqrt # 只引入其中一个函数，用的时候不带模块名
5 print(sqrt(16)) # 4.0
6
7 import numpy as np # 引入并起个短名字，长模块名常这么写
```

第一种最稳妥，看到 `math.sqrt` 就知道这个函数是从哪来的。第二种写起来短，但引多了容易和自己的函数重名。第三种给长名字起简称，社区有约定俗成的简称，比如 `numpy` 简称 `np`，见到就照着写，别人一看就懂。

还有一种写法是 `from math import *` 星号，把模块里所有东西都引进来。不建议用，因为不知道引进了什么，很容易把自己的变量覆盖掉。

自己写的文件也能这样引。同一个目录下有个 `tools.py`，里面定义了 `add` 函数，那么在另一个文件里写 `import tools` 再用 `tools.add` 就能调用。这就是把代码分成多个文件管理的方式。

5.3 标准库，装完 Python 就有的工具

本节任务：知道有哪些常用的现成模块，需要时想得起来。

Python 自带一大批模块，不用另外安装，统称**标准库**。常用的如表 5.1。

表 5.1: 几个常用的标准库模块。

模块	干什么
math	数学函数，开方、取整、圆周率
random	随机数、随机抽样、打乱顺序
datetime	日期和时间的计算与格式化
collections	好用的容器，比如上面用的 Counter
json	读写 JSON 格式的数据
pathlib	处理文件路径
re	正则表达式，按模式查找和切分文字

举几个立刻能用上的例子：

```
1 import random
2 print(random.choice(["石头", "剪刀", "布"])) # 随机挑一个
3 nums = [1, 2, 3, 4, 5]
4 random.shuffle(nums) # 原地打乱
5 print(random.sample(range(1, 34), 6)) # 不重复地抽六个
6
7 from datetime import date
8 today = date.today()
9 print(today) # 2026-07-21 这样的格式
10 print(today.year, today.month)
11
12 from collections import Counter
13 print(Counter("hello world").most_common(3)) # 数字符出现次数
```

不用背，知道有这些东西存在就行。需要做某件事时先想想标准库里有没有现成的，搜一下往往就有。

5.4 装别人写的库

本节任务：会用 pip 安装第三方库。

标准库之外还有海量的**第三方库**，需要自己装。装库的工具叫 **pip**，跟着 Python 一起装好了。在命令行里执行：

```
pip install requests
```

几秒后就装好了，之后 import requests 就能用。几个常用命令：

```
pip install 包名 装一个包
pip install 包名==1.2.3 装指定版本
pip uninstall 包名 卸载
pip list 看已经装了哪些
```

国内直接装可能很慢甚至失败，可以换成国内镜像下载：

★核心

```
pip install requests -i https://pypi.tuna.tsinghua.edu.cn/simple
```

后面几章要用到的 `numpy` 和 `pandas` 就是这样装的。装之前先想清楚是不是真需要，标准库能解决的事没必要引入新依赖。

5.5 虚拟环境，别让项目互相打架

本节任务：理解为什么要给每个项目单独准备一套环境。

所有库都装在一起会有个麻烦。项目甲要某个库的老版本，项目乙要新版本，装了新的甲就跑不了。**虚拟环境**就是给每个项目单独开一个干净的空间，各装各的，互不影响。

创建和使用只有三步：

```
python -m venv .venv 在当前目录建一个虚拟环境
.venv\Scripts\activate 激活，Windows 用这行
source .venv/bin/activate 激活，macOS 和 Linux 用这行
```

激活之后命令行提示符前面会多出一个环境名字，这时 `pip install` 装的东西都只属于这个项目。用 `deactivate` 可以退出。

刚入门可以先不用，但知道有这回事。等你同时做两三个项目、或者跟着别人的项目跑代码时，它能省掉很多莫名其妙的报错。项目里常见的 `requirements.txt` 文件就是记录该项目需要哪些库，别人拿到后一行命令就能装齐：

```
pip install -r requirements.txt
```

5.6 小结

这一章学会了站在别人的肩膀上。一个 `py` 文件就是一个模块，用 `import` 引进来，三种写法各有适用场合。标准库装完 Python 就有，`math` `random` `datetime` `collections` `json` `pathlib` `re` 这几个最常用。标准库之外的第三方库用 `pip` 装，国内建议配镜像源。虚拟环境给每个项目单独一套依赖，避免版本打架。下一章处理文件，把程序算出来的东西存下来。

本章知识点

- **模块** —— 一个 py 文件就是一个模块，可以被别的文件引用
- **import** —— 引入整个模块，用的时候带模块名，最稳妥
- **from import** —— 只引入其中某个函数，写起来短但容易重名
- **as** —— 给模块起个短名字，长模块名常这么写
- **标准库** —— 装完 Python 就有的一批模块，不用另外安装
- **Counter** —— collections 里的计数容器，配 most_common 直接拿高频项
- **第三方库** —— 标准库之外的库，需要自己装
- **pip** —— 装库的工具，install 装、uninstall 卸、list 看已装
- **镜像源** —— 国内下载慢时换个源，加 -i 参数
- **虚拟环境** —— 给每个项目单独一套依赖，避免版本互相打架
- **requirements.txt** —— 记录项目需要哪些库，一行命令装齐